

AIPL Runtime Feature Matrix

7 implementations compared

Yasushi Kodama

2026-05-13

Overview

AIPL (Actor-based Intelligent Parallel Language, formerly ABCL/c+) has grown into a **seven-runtime project**. This document provides a side-by-side comparison of every language and infrastructure feature across all implementations, grounded in source surveys.

Legend. ✓ = full support, △ = partial, ✕ = not supported, – = not applicable.

	Short name	Source
The seven runtimes.	Python (annotated)	src/python-aipl/
	Python (inferred)	src/python-aipl-inferred/
	OCaml	src/*.ml, _build/.../repl_thread.exe
	JS-OCaml (server)	src/app.js, console_server.js, ide.js + OCaml web_gateway
	JS-Browser (serverless)	src/browser-abcl/
	JS-Node (server)	src/node-aipl-server/ (Node HTTP server wrapping browser-abcl)
	C (abcl2c)	src/abcl2c.ml codegen + src/abcl_gui_runtime.c

Column abbreviations in matrices below. **Py-A** = Python annotated, **Py-I** = Python inferred, **OCaml**, **JS-O** = JS via OCaml backend, **JS-B** = JS in browser, **JS-N** = JS in Node server, **C** = abcl2c codegen.

Inheritance relationships.

- **JS-OCaml** is a thin HTTP client over the OCaml backend, so its feature set inherits OCaml's (plus the new `/api/typecheck` endpoint).
- **JS-Node** wraps the same browser-abcl parser / type-checker / interpreter inside a Node.js HTTP server. Runtime feature set therefore mirrors JS-Browser.
- **Python (inferred)** imports parser, interpreter, AI, remote, and dashboard from `../python-aipl/`. All *runtime* features match Python (annotated); only the *static type-checker* layer differs (full Hindley-Milner instead of Phase 11+ annotation-driven).

Sample programs

Number of `.abcl` sample programs reachable by each runtime (core directory + AI integration + remote-actor demos):

	Py-A	Py-I	OCaml	JS-O	JS-B	JS-N	C
core samples	33	33	57	57	5	5	57
AI integration samples	11	11	8	8	0	0	8
remote-actor samples	8	8	1	1	0	0	1
total	52	52	66	66	5	5	66

Cross-cutting / not tied to a specific runtime:

- `aipl-self-host/`: 48 AIPL-in-AIPL self-host programs
- `docker/cross/samples/`: 4 cross-language interop samples

Note that the C runtime processes the same `.abcl` files as OCaml via `abcl2c`; of the 66 reachable samples, 48 currently pass the `abcl2c` smoke test (the remaining 9 in `abclc/` have pre-existing type ambiguities surfaced by our hard-fail policy).

Core language

Feature	Py-A	Py-I	OCaml	JS-O	JS-B	JS-N	C
method injection (<code>add_method/remove_method</code>)	✓	✓	×	×	×	×	×
<code>now</code> synchronous send	✓	✓	✓	✓	✓	✓	✓
<code>future</code> async send	✓	✓	✓	✓	✓	✓	✓
<code>await</code> future block	✓	✓	✓	✓	✓	✓	✓
<code>send</code> fire-and-forget	✓	✓	✓	✓	✓	✓	✓
<code>become</code> actor class swap	✓	✓	✓	✓	×	×	×
<code>select</code> selective receive	✓	✓	✓	✓	✓	✓	×
top-level functions	✓	✓	△	△	×	×	✓
signatures / overloads	×	×	✓	✓	×	×	×
dynamic compile (<code>compile()</code>)	✓	✓	×	×	×	×	×
worker pool / <code>DynamicWorkerPool</code>	✓	✓	×	×	△	△	×

Type system

Feature	Py-A	Py-I	OCaml	JS-O	JS-B	JS-N	C
type inference	△trace	✓HM	✓HM	✓HM	✓flow	✓flow	✓HM+spec
type annotations	✓	△ignored	×	×	×	×	×
records <code>{a:int, b:str}</code>	✓	✓	△type only	×	×	×	△type only
tuples <code>(1, "a")</code>	✓	✓	×	×	×	×	×
arrays (typed, multi-dim)	✓	✓	✓	✓	△	△	△
generics on functions	×	✓Forall	△Forall	△	×	×	×

Phase 11+ advanced features

Python (inferred) runs HM only, so the annotation-driven Phase 11+ static checks that Python (annotated) performs are not re-implemented. Programs still execute at runtime via the shared interpreter, but the static guarantees from Phase 12 / 14 / 15 / 16 are lost. Phase 13 (channels) and Phase 17 (`scope`) are runtime features, so they remain available.

Feature	Py-A	Py-I	OCaml	JS-O	JS-B	JS-N	C
channels (CSP-style) —Phase 13	✓	✓	×	×	×	×	×
linear types / use-after-move —Phase 14	✓	×	×	×	×	×	×
owned / pub fields —Phase 15	✓	×	×	×	×	×	×
effects <code>!{fs,ai,net,mut}</code> —Phase 12	✓	×	×	×	×	×	×
structured concurrency <code>scope</code> —Phase 17	✓	✓	×	×	×	×	×
transient cast at any-boundary —Phase 16	✓	×	×	×	×	×	×

Networking, AI, infrastructure

Feature	Py-A	Py-I	OCaml	JS-O	JS-B	JS-N	C
remote actors (HTTP)	✓	✓	△no WS	△	×	△srv only	✓via OCaml
WebSocket	×	×	×	×	×	×	×
AI integration (<code>ai_call</code>)	✓stream/img	✓	✓	✓	✓mock	✓mock	✓
AI governance (budget/concurrent/fallback)	✓	✓	✓	✓	×	×	✓
HMAC-signed remote send	✓	✓	×	×	×	×	✓
persistent actor state	✓	✓	×	×	×	×	×
live dashboard (SSE)	✓	✓	△polling	△	×	×	✓
<code>/api/typecheck</code> JSON endpoint	×	×	✓	✓	×	✓	—
<code>/api/run</code> JSON endpoint	×	×	×	×	×	✓	—

C-version-specific codegen targets

Target	C
C + pthread standalone binary	✓
C + SDL2 GUI binary (1178-line runtime)	✓
Xinu embedded-OS target (<code>--xinu</code>)	✓
Python target (<code>--python</code>)	✓

Key observations

Python (annotated) is the most feature-complete runtime

Of the 30 surveyed features, Python (annotated) ticks roughly 28. It owns the full Phase 11–17 stack (channels, linear types, owned fields, effects, structured concurrency, transient casts), method injection, dynamic compile, persistent actor state, and full AI integration (including streaming and images).

Python (inferred) is Python’s HM-typed twin

Runtime features match Python (annotated) exactly because the interpreter is shared. The trade is on the static side: the Phase 11+ annotation-driven checks are replaced with a Hindley-Milner inference pass. Inferred types cross-verified against OCaml: on shared samples (`Hello.abcl`, `counter.abcl`) the two runtimes produce identical type schemes.

OCaml is the canonical core

Implements roughly 15 of the surveyed features. Phase 11+ exist only as `.abcl` design-document samples, not implemented. Solid `become` / `select` / HM inference; remote is HTTP only (no HMAC, no WebSocket).

JS-OCaml $\approx$ OCaml

Thin HTTP client over the OCaml backend, so features inherit. The only differentiator is the `/api/typecheck` endpoint added in commit 5227f87.

JS-Browser is the minimal implementation

Around 8 of the 30 features. Basic actor model plus our newly-added flow-sensitive type inference. No `become`, no channels, no remote. AI integration exists but is mock-only.

JS-Node (Node-server, newest)

Wraps the browser-abcl parser, type-checker, and interpreter inside a Node.js HTTP server. Exposes `/api/typecheck` (the same JSON shape as OCaml’s endpoint) and `/api/run` (executes and returns stdout) —the only runtime with the latter. Runtime feature set mirrors browser-abcl; the value-add is having both the type-checker and the interpreter reachable over HTTP without bringing up either OCaml or a browser.

C version — surprisingly strong on infrastructure

About 17 of 30. `become` and `select` are not implemented (codegen emits `/* unsupported */`), but it inherits OCaml’s HMAC, SSE, AI governance, and adds **three additional codegen targets** (SDL2, Xinu, Python). Fields, parameters, and locals are specialized to native C types.

Method injection — a Python-family exclusive

Both Python variants use mutable method dispatch tables that can be mutated at runtime via `add_method` / `remove_method`. Other runtimes substitute `become` (whole-class swap) as the closest equivalent. JS-Browser and JS-Node lack even `become` (pure actor execution).

Type inference cross-verification

For shared samples (`abclc/Hello.abcl`, `abclc/counter.abcl`), the three HM-based runtimes (**OCaml**, **Python-inferred**, **C**) produce identical inferred types:

```
Hello:  count : float
        init  : (int) -> unit
        greet : () -> unit
        inc   : () -> unit
```

```
Counter: count : float
        inc   : () -> unit
        dec   : (float) -> unit    <- monomorphized via call site
```

The flow-sensitive variants (**JS-Browser**, **JS-Node**) reach the same answers on shared samples within the limits of their algorithm —field types match, but method signature schemes are not produced the same way.

Phase 11+ implementation gap

The full Phase 11–17 stack is implemented only in Python (annotated). This represents AIPL’s language-evolution frontier; Python (annotated) serves as the research runtime. Python (inferred) keeps the runtime side but trades the static checks for HM-style polymorphic inference — a different research axis (declarative typing vs. annotated capabilities).

Generated 2026-05-13. Includes `python-aipl-inferred` (commit `270f291`) and `node-aipl-server` (commit `adeb0b6`). For source pointers, run `grep` against the files listed in each runtime’s source column.